VARIABLES IN JAVA

What is a Variable?

Variables are containers to store data in memory. Each variable has a name, type and value. It is the basic unit of storage in a program. Java has 4 types of variables.

Example:

int age = 20;

- int → data type
- age → variable name
- 20 → value stored in memory

Variable Memory Representation

Imagine memory like this:

Variable Name → age
Value → 20
Memory Location → stored in RAM

Example diagram:

age

```
┌────┐
│ 20 │
└────┘
```

Syntax of Variable Declaratio

DataType variableName = value;

Example:

```
int number = 10;
double price = 99.99;
char grade = 'A';
String name = "Omkar";
```

Steps of Creating a Variable in Java

There are three main steps.

1 Declaration

Declaring a variable means telling Java what type of data it will **store**.

Example:

int age;

Here:

- int → data type
- age → variable name

2 Initialization

Assigning a value to a variable.

Example:

age = 25;

3 Declaration + Initialization Together

Example:

int age = 25;

Types of Variables in Java

Java variables are mainly divided into three types.

1 Local Variables
2 Instance Variables
3 Static Variables

1 Local Variables :

Declared inside a method, constructor, or block. Accessible only within that block.

Scope of Local Variables :

The scope of a local variable means the area where it can be used.

A local variable can be used only inside the method or block where it is declared.

Example Program :

```
public class LocalVariableExample {

   public static void main(String[] args) {

      int number = 10;        // Local variable

      System.out.println(number);

   }
}
```

Explanation

In this example :

int number = 10;

- int → data type
- number → local variable name
- 10 → value stored in the variable

The variable number is declared inside the main() method, so it is a local variable.

Memory Representation :

Local variables are stored in **stack memory**.

Example:

main() Method Stack

number

```
┌──────┐
│ 10   │
└──────┘
```

When the main method finishes execution, the variable is removed from memory.

Local Variable Life Cycle :

Method Starts
↓
Local Variable Created
↓
Method Executes

↓

Method Ends

↓

Local Variable Destroyed

Example Showing Local Variable Lifecycle :

```
public class Example {

   public static void main(String[] args) {

      int a = 10;

      System.out.println(a);
   }
}
```

Lifecycle:

main() starts

↓

variable a created

↓

value 10 stored

↓

method finishes

↓

variable a destroyed

Characteristics of Local Variables :

- Declared inside methods, constructors, or blocks
- Accessible only within the method or block
- Stored in stack memory

- Must be initialized before use
- Created when method starts execution
- Destroyed when method execution ends

Real-Life Example :

Think of a local variable like a temporary notebook used during a meeting.

Example:

Meeting starts → notebook created
Notes written → used during meeting
Meeting ends → notebook discarded

Similarly:

Method starts → local variable created
Method runs → variable used
Method ends → variable removed

2 Instance Variables :

Declared inside a class but outside any method. Each object of the class has its own copy.

It is called an instance variable because each object (instance) of the class gets its own copy of the variable.

Example Program :

```
class Student {

   int id;
   String name;
}
```

Explanation :

In this class:

```
int id;
String name;
```

- id → instance variable
- name → instance variable

These variables belong to objects of the Student class.

Creating Objects :

```
Student s1 = new Student();
Student s2 = new Student();
```

Now two objects are created:

s1
s2

Each object has its own copy of instance variables.

Memory Representation :

Object s1

id = 101
name = Omkar

Object s2

id = 102
name = Rahul

Even though both objects belong to the same class, their data can be different.

Complete Example Program :

```
class Student {

  int id;
  String name;

  public static void main(String[] args) {

    Student s1 = new Student();
    Student s2 = new Student();

    s1.id = 101;
    s1.name = "Omkar";

    s2.id = 102;
    s2.name = "Rahul";
```

```
        System.out.println(s1.id + " " + s1.name);
        System.out.println(s2.id + " " + s2.name);
    }
}
```

Output :

101 Omkar
102 Rahul

Explanation:

- s1 stores its own values
- s2 stores different values

Both objects have separate instance variable data.

Where Instance Variables Are Stored :

Instance variables are stored in Heap Memory.

Memory structure:

Heap Memory

Student Object s1
---
id = 101
name = Omkar

Student Object s2
---
id = 102
name = Rahul

Every time we create a new object using new, Java allocates memory in the heap.

Accessing Instance Variables :

Instance variables are accessed using objects.

Syntax:

objectName.variableName

Example:

```
Student s1 = new Student();

s1.id = 101;
s1.name = "Omkar";
```

Here:

s1 → object
id → instance variable

Life Cycle of Instance Variables

Object Created
↓
Instance Variable Created
↓
Object Used
↓
Object Destroyed
↓
Instance Variables Removed

Instance variables exist as long as the object exists.

Characteristics of Instance Variables

- Declared inside class but outside methods
- Each object has its own copy
- Stored in heap memory
- Accessed using object reference
- Can have default values
- Created when object is created

Real Life Example

Think of a Student class.

Each student has different information.

Student 1
Name = Omkar
Age = 22

Student 2
Name = Rahul
Age = 21

Here:

Name → instance variable
Age → instance variable

Each student object stores different values.

Differentiate between instance and local variables

Instance Variable	Local Variable
Declared outside the method, directly invoked by the method.	Declared within the method.
Has a default value.	No default value
It can be used throughout the class.	The scope is limited to the method.

3 Static Variables :

the same copy of the static variable. A Static Variable is a variable that is declared inside a class using the static keyword.

Unlike instance variables, static variables belong to the class itself, not to individual objects.

This means all objects of the class share

Syntax

static dataType variableName;

Example:

static String college;

Here:

- static → keyword
- String → data type
- college → variable name

Example Program

```
class Student {

  int id;
  String name;
  static String college = "ABC College";

  public static void main(String[] args) {

    Student s1 = new Student();
    Student s2 = new Student();

    s1.id = 101;
```

```
        s1.name = "Omkar";

        s2.id = 102;
        s2.name = "Rahul";

        System.out.println(s1.id + " " + s1.name + " " + s1.college);
        System.out.println(s2.id + " " + s2.name + " " + s2.college);

    }
}
```

Output

101 Omkar ABC College
102 Rahul ABC College

Explanation:

- id and name are **instance variables**
- college is a **static variable**

Both objects **share the same college value**.

Memory Representation

Class: Student

Static Variable (Shared)
college = ABC College

Objects:

Object s1
id = 101
name = Omkar

Object s2

id = 102
name = Rahul

Important point:

college → only ONE copy exists

Why Static Variables Are Used

Static variables are used when **a value should be common for all objects**.

Example situations:

- College name for all students
- Company name for all employees
- Country name
- Interest rate in a bank system

Example:

static String country = "India";

Every object will use the **same country value**.

Accessing Static Variables

Static variables can be accessed in two ways.

1 .Using Object

s1.college

2 . Using Class Name (Best Practice)

Student.college

Example:

System.out.println(Student.college);

Using the class name is recommended.

Example Using Class Name

```
class Test {

   static int count = 0;

   public static void main(String[] args) {

      System.out.println(Test.count);

   }
}
```

Output:

0

Static Variable Memory Location

Static variables are stored in the Method Area (Class Area) of memory.

Memory structure:

Method Area

Student Class
---
college = ABC College

Objects:

Heap Memory

Student Object s1
id
name

Student Object s2

id
name

So:

Static variables → Method Area
Instance variables → Heap
Local variables → Stack

Life Cycle of Static Variables

Class Loaded
↓
Static Variable Created
↓
Program Execution
↓
Program Ends
↓
Static Variable Destroyed

Static variables exist as long as the class is loaded in memory.

Characteristics of Static Variables

- Declared using the static keyword
- Belong to the class, not objects
- Only one copy exists
- Shared by all objects
- Stored in method area memory
- Can be accessed using class name

Example Showing Shared Variable

```
class Counter {

   static int count = 0;

   Counter() {

      count++;

      System.out.println(count);
   }
   public static void main(String[] args) {

      Counter c1 = new Counter();
      Counter c2 = new Counter();
      Counter c3 = new Counter();
   }
}
```

Output:

1
2
3

Explanation:

Each object increases the same static variable.

Difference Between Instance and Static Variables

Feature	Instance Variable	Static Variable
Belongs to	Object	Class
Copies	Multiple copies	Single copy
Memory	Heap	Method Area
Access	Object	Class name

Summary

Static variables:

- Are declared using the static keyword
- Belong to the class rather than objects
- Have only one shared copy
- Are stored in the method area
- Used when data should be common for all object